Programación funcional con Isabelle/HOL

Tema 4 — Razonamiento Automático

Isabelle/HOL

Contenido

Isabelle también es un lenguaje de programación

Contenido

Isabelle también es un lenguaje de programación

Tipos básicos

Contenido

Isabelle también es un lenguaje de programación

Tipos básicos

Definiciones

Contenido

Isabelle también es un lenguaje de programación

Tipos básicos

Definiciones

Estructuras de datos

Contenido

Isabelle también es un lenguaje de programación

Tipos básicos

Definiciones

Estructuras de datos

Tipos de datos y recursión

Contenido

Isabelle también es un lenguaje de programación

Tipos básicos

Definiciones

Estructuras de datos

Tipos de datos y recursión

Resumen

Isabelle también es un lenguaje de programación

Un cambio de perspectiva

- En la Parte I demostramos propiedades sobre fórmulas proposicionales mediante las reglas proposicionales de deducción natural.

Un cambio de perspectiva

- En la Parte I demostramos propiedades sobre fórmulas proposicionales mediante las reglas proposicionales de deducción natural.
- Ahora vamos a escribir programas dentro de Isabelle.

Un cambio de perspectiva

- En la Parte I demostramos propiedades sobre fórmulas proposicionales mediante las reglas proposicionales de deducción natural.
- Ahora vamos a escribir programas dentro de Isabelle.
- Y en las próximas dos semanas, a demostrar que son correctos: no probándolos con ejemplos, sino para todas las entradas.

Un cambio de perspectiva

- En la Parte I demostramos propiedades sobre fórmulas proposicionales mediante las reglas proposicionales de deducción natural.
- Ahora vamos a escribir programas dentro de Isabelle.
- Y en las próximas dos semanas, a demostrar que son correctos: no probándolos con ejemplos, sino para todas las entradas.

El lenguaje de Isabelle es funcional, muy parecido a Haskell:
funciones puras, sin variables que cambian de valor, y tipos que Isabelle deduce solo.

El comando `value`: evaluar una expresión

```
value "Suc 0"
```

El comando `value`: evaluar una expresión

```
value "Suc 0"
```

- `value` calcula el valor de una expresión, como la consola de un intérprete.

El comando `value`: evaluar una expresión

```
value "Suc 0"
```

- `value` calcula el valor de una expresión, como la consola de un intérprete.
- En Isabelle/jEdit el resultado aparece en el panel de salida: `1`.

El comando `value`: evaluar una expresión

```
value "Suc 0"
```

- `value` calcula el valor de una expresión, como la consola de un intérprete.
- En Isabelle/jEdit el resultado aparece en el panel de salida: `1`.

Durante todo el tema: se escribe una definición, y se prueba con `value` antes de intentar demostrar nada sobre ella.

Tipos básicos

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Expresión	Resultado
`value "Suc 0"`	1

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Expresión	Resultado
`value "Suc 0"`	1
`value "(1::nat) + 2"`	3

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Expresión	Resultado
`value "Suc 0"`	1
`value "(1::nat) + 2"`	3
`value "(2::nat) * 3"`	6

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Expresión	Resultado
`value "Suc 0"`	1
`value "(1::nat) + 2"`	3
`value "(2::nat) * 3"`	6
`value "(7::nat) div 3"`	2
`value "(7::nat) mod 3"`	1

Los números naturales: `nat`

Se construyen con dos constructores, al estilo de Peano: `0` y `Suc` (el sucesor). El numeral `1` abrevia `Suc 0`.

Expresión	Resultado
`value "Suc 0"`	1
`value "(1::nat) + 2"`	3
`value "(2::nat) * 3"`	6
`value "(7::nat) div 3"`	2
`value "(7::nat) mod 3"`	1

`div` es el cociente entero y `mod` el resto.

El : : y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"
```

El : : y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"

*** Term to be evaluated
contains free dictionaries
```

El : : y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"
```

```
*** Term to be evaluated
contains free dictionaries
```

Con anotación:

```
value "length [1,2,3::nat]"
```

Resultado: 3

El : : y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"
```

```
*** Term to be evaluated
contains free dictionaries
```

Con anotación:

```
value "length [1,2,3::nat]"
```

Resultado: 3

- Los numerales están sobrecargados: 1 puede ser natural, entero, real...

El :: y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"
```

```
*** Term to be evaluated
contains free dictionaries
```

Con anotación:

```
value "length [1,2,3::nat]"
```

Resultado: 3

- Los numerales están sobrecargados: 1 puede ser natural, entero, real...
- Isabelle no sabe cuál elegir y no puede calcular. `::nat` se lo dice.

El :: y un error que va a ver

Sin anotación de tipo:

```
value "length [1,2,3]"
```

```
*** Term to be evaluated
contains free dictionaries
```

Con anotación:

```
value "length [1,2,3::nat]"
```

Resultado: 3

- Los numerales están sobrecargados: 1 puede ser natural, entero, real...
- Isabelle no sabe cuál elegir y no puede calcular. `::nat` se lo dice.

El mensaje es críptico, pero la causa casi siempre es esta: falta un `::` en algún numeral.

Enteros y booleanos

Enteros: `int`

```
value "(1::int) + -2"   -1
```

Los naturales no tienen negativos; los enteros sí.

Enteros y booleanos

Enteros: `int`

```
value "(1::int) + -2"   -1
```

Los naturales no tienen negativos; los enteros sí.

Booleanos: `bool`

```
value "True ∧ True"      True
value "True ∧ False"     False
value "True ∨ False"     True
value "¬True"            False
value "False ⟶ True"     True
```

Enteros y booleanos

Enteros: `int`

```
value "(1::int) + -2"   -1
```

Los naturales no tienen negativos; los enteros sí.

Booleanos: `bool`

```
value "True ∧ True"      True
value "True ∧ False"     False
value "True ∨ False"     True
value "¬True"            False
value "False ⟶ True"     True
```

Las conectivas de la Parte I son funciones sobre `bool`: se pueden evaluar.

Definiciones

Definir una función: `definition`

```
definition xor :: "bool ⇒ bool ⇒ bool" where
  "xor A B ≡ (A ∧ ¬B) ∨ (¬A ∧ B)"
```

Definir una función: `definition`

```
definition xor :: "bool ⇒ bool ⇒ bool" where
  "xor A B ≡ (A ∧ ¬B) ∨ (¬A ∧ B)"
```

- `bool ⇒ bool ⇒ bool`: recibe dos booleanos y devuelve uno.

Definir una función: `definition`

```
definition xor :: "bool ⇒ bool ⇒ bool" where
  "xor A B ≡ (A ∧ ¬B) ∨ (¬A ∧ B)"
```

- `bool ⇒ bool ⇒ bool`: recibe dos booleanos y devuelve uno.
- $\equiv$ se lee «se define como».

Definir una función: `definition`

```
definition xor :: "bool ⇒ bool ⇒ bool" where
  "xor A B ≡ (A ∧ ¬B) ∨ (¬A ∧ B)"
```

- `bool ⇒ bool ⇒ bool`: recibe dos booleanos y devuelve uno.
- $\equiv$ se lee «se define como».

```
lemma "xor True True = False"
by (simp add: xor_def)
```

Definir una función: `definition`

```
definition xor :: "bool ⇒ bool ⇒ bool" where
  "xor A B ≡ (A ∧ ¬B) ∨ (¬A ∧ B)"
```

- `bool ⇒ bool ⇒ bool`: recibe dos booleanos y devuelve uno.
- $\equiv$ se lee «se define como».

```
lemma "xor True True = False"
by (simp add: xor_def)
```

- Primera demostración sobre un programa. `simp` reescribe usando la definición, que se llama `xor_def`.

Que `simp` use la definición siempre

```
declare xor_def[simp]

lemma "xor True False = True"
by simp
```

Que `simp` use la definición siempre

```
declare xor_def[simp]

lemma "xor True False = True"
by simp
```

- `declare ...[simp]` añade la definición al conjunto de reglas que `simp` usa por su cuenta.

Que simp use la definición siempre

```
declare xor_def[simp]

lemma "xor True False = True"
by simp
```

- declare ...[simp] añade la definición al conjunto de reglas que simp usa por su cuenta.
- Ya no hace falta decirle add: xor_def.

Que `simp` use la definición siempre

```
declare xor_def[simp]

lemma "xor True False = True"
by simp
```

- `declare ...[simp]` añade la definición al conjunto de reglas que `simp` usa por su cuenta.
- Ya no hace falta decirle `add: xor_def`.

`simp` es un procedimiento de reescritura: sustituye iguales por iguales hasta que no puede seguir. En la semana 10 veremos cómo funciona por dentro.

Condicionales: `if` y `case`

```
definition absoluto :: "int ⇒ int" where
  "absoluto x ≡ (if x ≥ 0 then x else -x)"
```

Condicionales: `if` y `case`

```
definition absoluto :: "int ⇒ int" where
  "absoluto x ≡ (if x ≥ 0 then x else -x)"

value "absoluto(-3)"                                    ⇝ 3
```

Condicionales: `if` y `case`

```
definition absoluto :: "int ⇒ int" where
  "absoluto x ≡ (if x ≥ 0 then x else -x)"

value "absoluto(-3)"
```

⇝ 3

```
definition es_sucesor :: "nat ⇒ bool" where
  "es_sucesor n ≡ (case n of
    0     ⇒ False
  | Suc m ⇒ True)"
```

Condicionales: `if` y `case`

```
definition absoluto :: "int ⇒ int" where
  "absoluto x ≡ (if x ≥ 0 then x else -x)"

value "absoluto(-3)"                                    ⇝ 3
```

```
definition es_sucesor :: "nat ⇒ bool" where
  "es_sucesor n ≡ (case n of
    0     ⇒ False
  | Suc m ⇒ True)"

value "es_sucesor 3"                                    ⇝ True
```

Condicionales: `if` y `case`

```
definition absoluto :: "int ⇒ int" where
  "absoluto x ≡ (if x ≥ 0 then x else -x)"

value "absoluto(-3)"                                    ⇝ 3
```

```
definition es_sucesor :: "nat ⇒ bool" where
  "es_sucesor n ≡ (case n of
    0     ⇒ False
  | Suc m ⇒ True)"

value "es_sucesor 3"                                    ⇝ True
```

`case` distingue por la forma del dato: un natural es `0` o es `Suc` de otro. Esa idea es la base de todo lo que sigue.

`let` y funciones anónimas

Definiciones locales

```
value "let x = 3::nat in x * x"  9
```

`let` y funciones anónimas

Definiciones locales

```
value "let x = 3::nat in x * x"
```

9

Funciones anónimas con λ

```
value "(λx. x + x) 1::nat"
```

2

`let` y funciones anónimas

Definiciones locales

```
value "let x = 3::nat in x * x"  9
```

Funciones anónimas con λ

```
value "(λx. x + x) 1::nat"  2
```

$\lambda x.\ x + x$ es «la función que a x le asigna $x + x$», sin darle nombre. Es la misma λ de Python o de las lambdas de Java.

Estructuras de datos

Pares y listas

Pares: tipo producto `nat × nat`; `fst` y `snd` dan cada componente.

```
value "let p = (2,3)::nat × nat in fst p + 1 = snd p" ⇝ True
```

Pares y listas

Pares: tipo producto `nat × nat`; `fst` y `snd` dan cada componente.

```
value "let p = (2,3)::nat × nat in fst p + 1 = snd p" ⇝ True
```

Listas: todos los elementos del mismo tipo.

- `[]` es la lista vacía; `x#xs` añade `x` al principio de `xs`.

Pares y listas

Pares: tipo producto `nat × nat`; `fst` y `snd` dan cada componente.

```
value "let p = (2,3)::nat × nat in fst p + 1 = snd p" ⇝ True
```

Listas: todos los elementos del mismo tipo.

- `[]` es la lista vacía; `x#xs` añade `x` al principio de `xs`.
- `hd xs` es el primer elemento; `tl xs`, el resto; `length xs`, la longitud.

Pares y listas

Pares: tipo producto `nat × nat`; `fst` y `snd` dan cada componente.

```
value "let p = (2,3)::nat × nat in fst p + 1 = snd p"  ⇝ True
```

Listas: todos los elementos del mismo tipo.

- `[]` es la lista vacía; `x#xs` añade `x` al principio de `xs`.
- `hd xs` es el primer elemento; `tl xs`, el resto; `length xs`, la longitud.

```
value "a#(b#(c#[]))"          [a, b, c]
value "length [1,2,3::nat]"   3
```

Pares y listas

Pares: tipo producto `nat × nat`; `fst` y `snd` dan cada componente.

```
value "let p = (2,3)::nat × nat in fst p + 1 = snd p" ⇝ True
```

Listas: todos los elementos del mismo tipo.

- `[]` es la lista vacía; `x#xs` añade `x` al principio de `xs`.
- `hd xs` es el primer elemento; `tl xs`, el resto; `length xs`, la longitud.

```
value "a#(b#(c#[]))"          [a, b, c]
value "length [1,2,3::nat]"   3
```

`[a,b,c]` es solo una abreviatura de `a#(b#(c#[]))`. Una lista se construye elemento a elemento.

Registros

```
record punto =
  coordenada_x :: int
  coordenada_y :: int

definition pt :: punto where
  "pt ≡ ⦇coordenada_x = 3, coordenada_y = 7⦈"
```

Registros

```
record punto =
  coordenada_x :: int
  coordenada_y :: int

definition pt :: punto where
  "pt ≡ ⦇coordenada_x = 3, coordenada_y = 7⦈"

 value "coordenada_x pt"                                    3
```

Registros

```
record punto =
  coordenada_x :: int
  coordenada_y :: int

definition pt :: punto where
  "pt ≡ ⦇coordenada_x = 3, coordenada_y = 7⦈"

 value "coordenada_x pt"                                          3
 value "let pt2 = pt⦇coordenada_x:=4⦈ in coordenada_x pt2"        4
```

Registros

```
record punto =
  coordenada_x :: int
  coordenada_y :: int

definition pt :: punto where
  "pt ≡ ⦇coordenada_x = 3, coordenada_y = 7⦈"

 value "coordenada_x pt"                                          3
 value "let pt2 = pt⦇coordenada_x:=4⦈ in coordenada_x pt2"        4
```

Como un `struct`. Pero `pt⦇coordenada_x:=4⦈` no modifica `pt`: construye un punto nuevo. En un lenguaje funcional nada cambia de valor.

Tipos de datos y recursión

Definir un tipo de datos: `datatype`

```
datatype 'a Lista = Vacia | Cons 'a "'a Lista"
```

Definir un tipo de datos: `datatype`

```
datatype 'a Lista = Vacia | Cons 'a "'a Lista"
```

- Una lista es o bien `Vacia`, o bien `Cons x xs`: un elemento seguido de otra lista.

Definir un tipo de datos: `datatype`

```
datatype 'a Lista = Vacia | Cons 'a "'a Lista"
```

- Una lista es o bien `Vacia`, o bien `Cons x xs`: un elemento seguido de otra lista.
- `Vacia` y `Cons` son los constructores: la única forma de fabricar una lista.

Definir un tipo de datos: `datatype`

```
datatype 'a Lista = Vacia | Cons 'a "'a Lista"
```

- Una lista es o bien `Vacia`, o bien `Cons x xs`: un elemento seguido de otra lista.
- `Vacia` y `Cons` son los constructores: la única forma de fabricar una lista.
- `'a` es una variable de tipo: hay listas de naturales, de booleanos, de lo que sea.

Definir un tipo de datos: `datatype`

```
datatype 'a Lista = Vacia | Cons 'a "'a Lista"
```

- Una lista es o bien `Vacia`, o bien `Cons x xs`: un elemento seguido de otra lista.
- `Vacia` y `Cons` son los constructores: la única forma de fabricar una lista.
- `'a` es una variable de tipo: hay listas de naturales, de booleanos, de lo que sea.

Es la lista enlazada de un curso de estructuras de datos, dicha en una línea. Isabelle ya trae su propia lista (`[]` y `#`); aquí la definimos para ver cómo se hace.

Una función recursiva: `fun`

```
fun conc :: "'a Lista ⇒ 'a Lista ⇒ 'a Lista" where
  "conc Vacia ys        = ys"
| "conc (Cons x xs) ys = Cons x (conc xs ys)"
```

Una función recursiva: `fun`

```
fun conc :: "'a Lista ⇒ 'a Lista ⇒ 'a Lista" where
  "conc Vacia ys        = ys"
| "conc (Cons x xs) ys = Cons x (conc xs ys)"
```

- Una ecuación por cada constructor: así se cubren todas las listas posibles.

Una función recursiva: `fun`

```
fun conc :: "'a Lista ⇒ 'a Lista ⇒ 'a Lista" where
  "conc Vacia ys        = ys"
| "conc (Cons x xs) ys = Cons x (conc xs ys)"
```

- Una ecuación por cada constructor: así se cubren todas las listas posibles.
- La segunda ecuación llama a `conc` sobre `xs`, que es más corta: por eso la recursión termina.

Una función recursiva: `fun`

```
fun conc :: "'a Lista ⇒ 'a Lista ⇒ 'a Lista" where
  "conc Vacia ys        = ys"
| "conc (Cons x xs) ys = Cons x (conc xs ys)"
```

- Una ecuación por cada constructor: así se cubren todas las listas posibles.
- La segunda ecuación llama a `conc` sobre `xs`, que es más corta: por eso la recursión termina.

Isabelle comprueba por su cuenta que la definición termina. Si no puede comprobarlo, rechaza la función.

Cómo se evalúa `conc`

```
conc (Cons a (Cons b Vacia)) (Cons c Vacia)
```

Cómo se evalúa `conc`

```
    conc (Cons a (Cons b Vacia)) (Cons c Vacia)
=   Cons a (conc (Cons b Vacia) (Cons c Vacia))   2.ª ecuación
```

Cómo se evalúa `conc`

```
    conc (Cons a (Cons b Vacia)) (Cons c Vacia)
=   Cons a (conc (Cons b Vacia) (Cons c Vacia))   2.ª ecuación
=   Cons a (Cons b (conc Vacia (Cons c Vacia)))   2.ª ecuación
```

Cómo se evalúa `conc`

```
    conc (Cons a (Cons b Vacia)) (Cons c Vacia)
=   Cons a (conc (Cons b Vacia) (Cons c Vacia))   2.ª ecuación
=   Cons a (Cons b (conc Vacia (Cons c Vacia)))   2.ª ecuación
=   Cons a (Cons b (Cons c Vacia))                1.ª ecuación
```

Cómo se evalúa `conc`

```
    conc (Cons a (Cons b Vacia)) (Cons c Vacia)
=   Cons a (conc (Cons b Vacia) (Cons c Vacia))   2.ª ecuación
=   Cons a (Cons b (conc Vacia (Cons c Vacia)))   2.ª ecuación
=   Cons a (Cons b (Cons c Vacia))                1.ª ecuación
```

Evaluar es reescribir con las ecuaciones de la definición.

Es exactamente lo que hace `simp`. Por eso las mismas ecuaciones sirven para calcular y para demostrar.

Recursión sobre los naturales

```
fun suma :: "nat ⇒ nat" where
  "suma 0       = 0"
| "suma (Suc m) = (Suc m) + suma m"

fun sumaImpares :: "nat ⇒ nat" where
  "sumaImpares 0       = 0"
| "sumaImpares (Suc n) = (2 * (Suc n) - 1) + sumaImpares n"
```

Recursión sobre los naturales

```
fun suma :: "nat ⇒ nat" where
  "suma 0       = 0"
| "suma (Suc m) = (Suc m) + suma m"

fun sumaImpares :: "nat ⇒ nat" where
  "sumaImpares 0       = 0"
| "sumaImpares (Suc n) = (2 * (Suc n) - 1) + sumaImpares n"

 value "suma 3"          6
 value "sumaImpares 3"   9
```

Recursión sobre los naturales

```
fun suma :: "nat ⇒ nat" where
  "suma 0       = 0"
| "suma (Suc m) = (Suc m) + suma m"

fun sumaImpares :: "nat ⇒ nat" where
  "sumaImpares 0       = 0"
| "sumaImpares (Suc n) = (2 * (Suc n) - 1) + sumaImpares n"

value "suma 3"          6
value "sumaImpares 3"   9
```

Mismo patrón que con listas: una ecuación para `0` y otra para `Suc`, que son los dos constructores de `nat`.

Una pregunta para la semana que viene

```
value "sumaImpares 1"  1
value "sumaImpares 2"  4
value "sumaImpares 3"  9
```

Una pregunta para la semana que viene

```
value "sumaImpares 1"  1
value "sumaImpares 2"  4
value "sumaImpares 3"  9
```

Parece que `sumaImpares n = n * n`.

Una pregunta para la semana que viene

```
value "sumaImpares 1"  1
value "sumaImpares 2"  4
value "sumaImpares 3"  9
```

Parece que `sumaImpares n = n * n`.

Con `value` se puede comprobar para 1, 2, 3..., pero no para todos los `n`.

Una pregunta para la semana que viene

```
value "sumaImpares 1"  1
value "sumaImpares 2"  4
value "sumaImpares 3"  9
```

Parece que `sumaImpares n = n * n`.

Con `value` se puede comprobar para 1, 2, 3..., pero no para todos los `n`.

Para eso hace falta demostrarlo: por inducción, en las próximas dos semanas.

Resumen

Lo que se vio

Construcción	Para qué

Lo que se vio

Construcción	Para qué
`value`	evaluar una expresión
`::`	fijar el tipo de un término

Lo que se vio

Construcción	**Para qué**
`value`	evaluar una expresión
`::`	fijar el tipo de un término
`definition`	definir una función sin recursión
`if`, `case`, `let`, λ	construir expresiones

Lo que se vio

Construcción	**Para qué**
`value`	evaluar una expresión
`::`	fijar el tipo de un término
`definition`	definir una función sin recursión
`if`, `case`, `let`, λ	construir expresiones
`record`	datos con campos
`datatype`	un tipo nuevo, dado por sus constructores
`fun`	una función recursiva, por ecuaciones

Lo que se vio

Construcción	**Para qué**
`value`	evaluar una expresión
`::`	fijar el tipo de un término
`definition`	definir una función sin recursión
`if`, `case`, `let`, λ	construir expresiones
`record`	datos con campos
`datatype`	un tipo nuevo, dado por sus constructores
`fun`	una función recursiva, por ecuaciones
`by simp`	demostrar reescribiendo con las definiciones

- Un tipo de datos se define por sus constructores, y una función recursiva tiene una ecuación por constructor.

- Un tipo de datos se define por sus constructores, y una función recursiva tiene una ecuación por constructor.
- Evaluar es reescribir con esas ecuaciones; `simp` hace lo mismo para demostrar.

- Un tipo de datos se define por sus constructores, y una función recursiva tiene una ecuación por constructor.
- Evaluar es reescribir con esas ecuaciones; `simp` hace lo mismo para demostrar.
- Si `value` falla con «free dictionaries», falta un `::` en algún numeral.

- Un tipo de datos se define por sus constructores, y una función recursiva tiene una ecuación por constructor.
- Evaluar es reescribir con esas ecuaciones; `simp` hace lo mismo para demostrar.
- Si `value` falla con «free dictionaries», falta un `::` en algún numeral.
- Probar ejemplos con `value` no es demostrar. Eso es lo que viene.